Universidad de Chile
Facultad de Ciencias Físicas y Matemáticas
Departamento de Ciencias de la Computación

Project Manual: Game of Life on CPU and GPU

Computación en GPU

Author: Matías Saavedra

Document: developer manual
Last updated: Sunday 28th June, 2026
Santiago de Chile

Resumen

This document is the developer manual for the project: three implementations of Conway's Game of Life — a sequential CPU baseline, a CUDA backend and an OpenCL backend — built to benchmark cells-evaluated-per-second and report the GPU speed-up over the CPU. It explains the architecture from first principles, states the single invariant that shapes every design decision (*the three backends must compute byte-for-byte identical boards so that any difference in speed is the real result*), and then walks the source code function by function, with the C++ files explained line by line. All three backends are complete: the CPU backend is the correctness oracle, and the CUDA and OpenCL backends are each verified bit-for-bit against it. The headline metric is fixed throughout: Mcells/s = (rows $\times$ cols $\times$ gens)/($t \times 10^6$), measured against a no-op renderer so that rendering and host $\leftrightarrow$ device transfers never pollute the number. A final section documents the real-time **Qt + OpenGL viewer** (`gol_gui`) built on top of the same engines, which displays the GPU-computed board through **CUDA $\leftrightarrow$ OpenGL interop** (zero host round trip), with a host-upload fallback that also lets it run on the CPU engine with no CUDA toolkit present.

Índice de Contenidos

Índice de Figuras

Índice de Tablas

Índice de Códigos

1. What this manual is, and how to read it

This is the engineering manual for the codebase, not the graded report. The graded report lives in `main.tex` and presents experiments and results; this manual explains *how the code works and why it is built the way it is*. Read it top to bottom the first time: the early sections establish the one constraint that everything else follows from, and the later sections assume you already understand it.

The manual blends two styles. It is structured like an engineering report (setup, mechanism, tables, explicit trade-offs) but written like a deep code explanation (every non-obvious decision is traced to *why it exists* and *what would break if it were different*). For the C++ translation units it descends to a line-by-line account; for the headers, which are mostly declarations, it summarises and shows the parts that carry real logic.

Build.

The manual compiles with the same vendored template as the report:

Código 1: Building this manual.

```
cd report && latexmk -pdf manual.tex
# or: pdflatex manual.tex && pdflatex manual.tex   (second pass for the TOC)
```

2. The problem, and the invariant it forces

The assignment looks like a simulation task but is really a *measurement* task. The deliverable is a number and an explanation of that number: how many cells per second each backend evaluates, and *why* the GPU wins (or loses) at a given grid size. Conway's Game of Life — here the HighLife variant `B36/S23` — is only the workload. It is deliberately a good workload for the question being asked: a nine-point stencil over one-byte cells is arithmetically trivial and memory-bound, so the experiment isolates memory bandwidth and launch/transfer overhead, which is exactly the interesting story for a GPU report.

Every architectural decision in the codebase serves one goal:

> *Run the identical computation on three backends and trust that any difference in output is a bug, so that any difference in speed is the real result.*

Two hard requirements fall out of that sentence, and they explain the entire design:

1. **Bit-for-bit equivalence across backends.** If CPU and GPU produce different boards, the cells/sec comparison is meaningless. This forces a *shared* cell layout, a *shared* rule, and *shared* edge handling.

2. **Measurement that is not polluted by I/O.** The headline metric is compute throughput, so rendering and host $\leftrightarrow$ device transfers must be removable from the timed path.

The rule variant is the one detail most likely to be implemented inconsistently, so it is worth stating precisely. It is *not* vanilla Conway:

- a dead cell is born on **exactly 3 OR exactly 6** live neighbours;
- a live cell survives on 2 or 3 live neighbours;
- everything else dies or stays dead.

The "birth on 6" clause is the easy mistake. It must hold identically in CPU, CUDA and OpenCL, which is why it is written exactly once (Section 4.2) and pinned by a dedicated test (Section 14).

3. Architecture at a glance

The codebase is a **strategy pattern**: a backend-agnostic core plus two swappable interfaces. The only thing that genuinely differs between backends — the per-generation compute — sits behind `ISimEngine`; output sits behind `IRenderer`; the application owns the loop and wires one of each at runtime from the parsed configuration.

Código 2: The strategy layout of the headless app.

```
Application (main.cpp) -- owns the loop + Config/CLI
 |-- ISimEngine   <- CpuEngine | CudaEngine | OpenCLEngine
 |-- IRenderer    <- NullRenderer | TextRenderer | AnsiRenderer
 '-- Grid + Pattern  (shared, backend-agnostic state & seed data)
```

The real-time Qt + OpenGL viewer (`gol_gui`) reuses the same `ISimEngine` backends but is a *separate* front-end rather than an `IRenderer` — Qt owns its event loop and the display path avoids the host `Grid` entirely. It is documented on its own in Section 12.

The dependency arrows point *from* engines *to* the core, never back — that one-directional dependency is what keeps the core backend-agnostic and lets a single host-side harness drive all three engines. The link graph is:

Código 3: Library dependency graph (CMake targets).

```
gol_core  (Grid, Config+CLI, patterns, RLE, interfaces)
  |-- gol_cpu     (CpuEngine; links Threads::Threads)
  |-- gol_render  (TextRenderer, AnsiRenderer)
  |-- gol_cuda    (CudaEngine; opt-in, -DBUILD_CUDA=ON)
  '-- gol_opencl  (OpenCLEngine; opt-in, -DBUILD_OPENCL=ON)
gol  executable  =  gol_core + gol_cpu + gol_render (+ gol_cuda / gol_opencl when built
    )
```

The UML class diagram in Figure 1 renders these relationships formally. The `Application` depends only on the two strategy interfaces; every backend and renderer is a realization behind them, and the shared `Grid` flows through all of them unchanged.

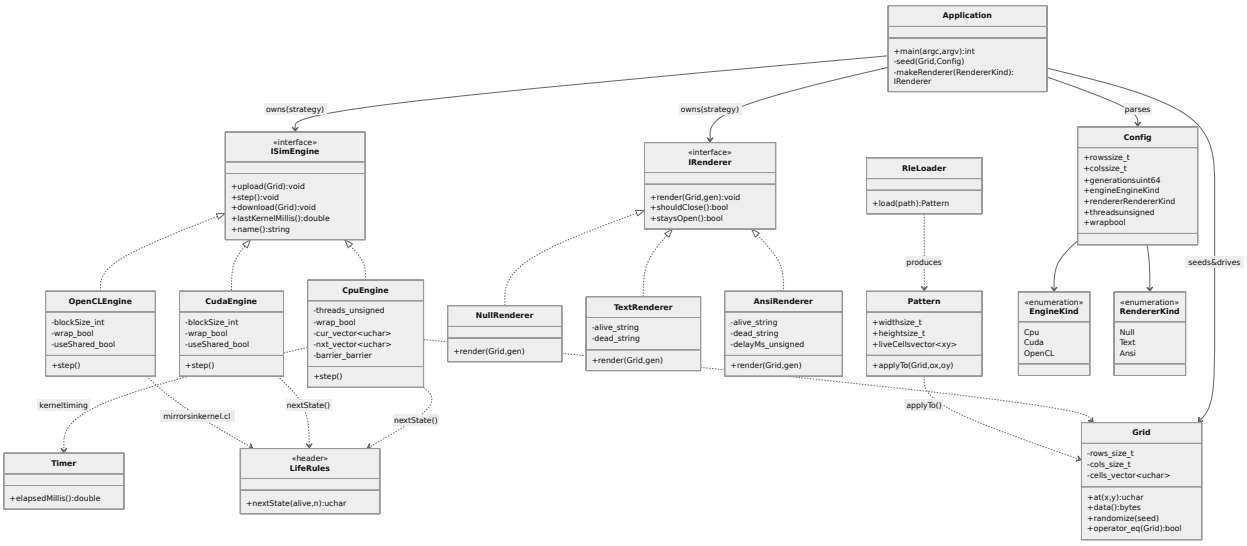

Figura 1: UML class diagram of the project (generated from `report/diagrams/architecture.mmd` with mermaid-cli). The `Application` owns one `ISimEngine` and one `IRenderer` — the two strategy interfaces. `CpuEngine`, `CudaEngine` and `OpenCLEngine` realize `ISimEngine`. `NullRenderer`, `TextRenderer` and `AnsiRenderer` realize `IRenderer`. Dashed arrows are dependencies: `CpuEngine` and `CudaEngine` use `LifeRules::nextState` directly, while `OpenCLEngine` mirrors that rule in `kernel.cl` (read at runtime); `RleLoader` produces a `Pattern` that stamps a `Grid`.

Table 1 is the file map. Header-only units carry no `.cpp` and impose no link dependency.

Tabla 1: File map and status. "HO" = header-only (logic lives entirely in the header). The "$\mathcal{S}$" column points to the section that explains it.

File	Responsibility	$\mathcal{S}$
include/gol/Grid.hpp	Board state; the shared layout (HO)	4.1
include/gol/LifeRules.hpp	The variant rule, written once (HO)	4.2
include/gol/Timer.hpp	Monotonic host stopwatch (HO)	4.3
include/gol/ISimEngine.hpp	Engine strategy interface (HO)	4.4
include/gol/IRenderer.hpp	Renderer strategy interface (HO)	4.4
include/gol/Config.hpp	Run configuration struct (HO)	4.5
include/gol/engines/CpuEngine.hpp	CPU engine declaration	7
include/gol/engines/CudaEngine.hpp	CUDA engine declaration	8
include/gol/engines/cuda/kernel.cuh	CUDA launcher declaration	8
include/gol/engines/OpenCLEngine.hpp	OpenCL engine declaration	9
include/gol/render/NullRenderer.hpp	No-op renderer (HO)	11
include/gol/render/TextRenderer.hpp	Text renderer declaration	11
include/gol/render/AnsiRenderer.hpp	In-place ANSI renderer decl.	11
include/gol/patterns/Pattern.hpp	Sparse seed pattern (HO)	10
include/gol/patterns/RleLoader.hpp	RLE loader declaration	10
src/core/Config.cpp	CLI parsing	5
src/core/main.cpp	The loop and the metric	6
src/engines/cpu/CpuEngine.cpp	CPU engine (seq + parallel)	7
src/engines/cuda/CudaEngine.cu	CUDA engine implementation	8
src/engines/cuda/kernel.cu	CUDA step kernels	8
src/engines/opencl/OpenCLEngine.cpp	OpenCL engine implementation	9
src/engines/opencl/kernel.cl	OpenCL kernels (read at runtime)	9
src/patterns/RleLoader.cpp	RLE decoding	10
src/patterns/Pattern.cpp	Stamping a pattern onto a grid	10
src/render/TextRenderer.cpp	ASCII frame dump	11
src/render/AnsiRenderer.cpp	In-place ANSI animation	11
include/gol/render/CudaGlInterop.hpp	CUDA $\leftrightarrow$ GL bridge decl.	12
src/render/CudaGlInterop.cu	CUDA $\leftrightarrow$ GL bridge (nvcc)	12
include/gol/gui/GolGlWidget.hpp	Viewport widget declaration	12
src/gui/GolGlWidget.cpp	Viewport: engine + interop + input	12
include/gol/gui/MainWindow.hpp	Window + control panel decl.	12
src/gui/MainWindow.cpp	Window, panel, signal wiring	12
src/gui/main_gui.cpp	Viewer entry point + CLI	12
src/gui/shaders/display.{vert,frag}	Display shaders (read at runtime)	12
scripts/run_gui.sh	Launch on the NVIDIA GPU (Optimus)	12
~~CMakeLists.txt~~	Build, GPU/GUI opt-in gating	13
tests/*.cpp	Executable spec of the invariants	14
scripts/sweep.sh	Benchmark sweeps $\rightarrow$ results/*.csv	15
analysis/results.ipynb	Plots the sweeps (Spanish)	15

4. The shared core

The core is everything that must be identical across backends, plus the glue that makes the backends interchangeable.

4.1. Grid — the spine of cross-backend equivalence

`Grid` is the single most important object in the project. It is a **flat, row-major `std::vector<unsigned char>`** where cell (x, y) lives at index $y \cdot \text{cols} + x$, with $0 =$ dead, $1 =$ alive.

Código 4: include/gol/Grid.hpp — the load-bearing parts.

```
class Grid {
public:
  Grid(std::size_t rows, std::size_t cols, unsigned char fillValue = 0)
      : rows_(rows), cols_(cols), cells_(rows * cols, fillValue) {}

  // Unchecked access by (x, y): the hot path must stay branch-free.
  unsigned char& at(std::size_t x, std::size_t y) { return cells_[y * cols_ + x]; }
  unsigned char  at(std::size_t x, std::size_t y) const { return cells_[y * cols_ + x]; }

  unsigned char* data() { return cells_.data(); }          // for host<->device transfers

  // Deterministic seeding: same seed => same board across all backends.
  void randomize(std::uint64_t seed, double aliveProbability = 0.3) {
    std::mt19937_64 rng(seed);
    std::bernoulli_distribution alive(aliveProbability);
    for (auto& c : cells_) c = alive(rng) ? 1u : 0u;
  }

  // Bit-for-bit comparison: the equivalence-test oracle check.
  bool operator==(const Grid& other) const {
    return rows_ == other.rows_ && cols_ == other.cols_ && cells_ == other.cells_;
  }
private:
  std::size_t rows_, cols_;
  std::vector<unsigned char> cells_; // row-major, size rows_ * cols_
};
```

Why this exact representation.

Each choice is forced by the equivalence requirement:

- **Flat and contiguous**, because a GPU device buffer is a flat array. A flat `unsigned char` buffer maps to `cudaMemcpy` / `clEnqueueWriteBuffer` with *zero* translation — the same bytes on host and device. A `vector<vector<»` would need a transform on every transfer, and the transform itself could introduce divergence.

- **One byte per cell, not one bit.** Bit-packing would be 8× smaller and use less bandwidth, but it would turn the neighbour read into a bit-extraction and make the three backends harder to keep identical. The project accepts the 8× memory cost to keep the stencil a plain array read. This is also *why* shared memory *hurts* rather than helps on the GPU (Section 15): the one-byte stencil's reuse is already served by L2, so explicit staging only adds a barrier and loses L1 reuse.
- **`at(x,y)` is unchecked.** The simulation hot path evaluates this billions of times; a bounds branch per access would be a measurable tax on a memory-bound kernel. Bounds are the caller's responsibility.

The invariant.

`Grid` owns no device memory and contains no device code. It is pure host state. Engines own their own ping-pong buffers and touch the `Grid` only through `data()` during `upload()`/`download()`. **What would break otherwise:** if the `Grid` held device pointers, the host harness and the equivalence test could no longer drive all three engines through one type.

Two methods exist purely for equivalence. `randomize` uses a seeded `std::mt19937_64`, so "same seed ⇒ same starting board" holds across backends — without a deterministic seed you could not start CPU and GPU from an identical state. `operator==` compares dimensions *and* the full byte vector; this is literally the oracle check the cross-backend test will call.

4.2. LifeRules — the rule written once, compiled three times

The rule variant is written exactly once and shared by source between the CPU and CUDA backends.

Código 5: include/gol/LifeRules.hpp — the single source of truth for the rule.

```
#ifdef __CUDACC__
#define GOL_FN __host__ __device__ inline
#else
#define GOL_FN inline
#endif

namespace gol {
// VARIANT (NOT vanilla Conway):
//   dead cell born on EXACTLY 3 OR EXACTLY 6 neighbours;
//   live cell survives on 2 or 3; everything else dies.
GOL_FN unsigned char nextState(unsigned char alive, int neighbors) {
  if (alive) {
    return (neighbors == 2 || neighbors == 3) ? 1u : 0u; // survive
  }
```

```
15   return (neighbors == 3 || neighbors == 6) ? 1u : 0u;   // birth (3 or 6)
16 }
17 } // namespace gol
```

The mechanism.

When the host C++ compiler reads this file, `nextState` is an ordinary `inline` function used by `CpuEngine`. When `nvcc` reads the *same* file it defines `___CUDACC___`, so the macro expands to `___host___ ___device___` and the CUDA kernel calls the byte-for-byte identical logic. CPU and CUDA therefore *cannot* diverge on the rule — they share source.

The acknowledged exception.

OpenCL C cannot `#include` a C++ header, so this function is mirrored in `kernel.cl` (read at runtime, kept diff-able against this header). The project deliberately does *not* chase zero duplication between CUDA and OpenCL: two duplicated copies that are easy to eyeball-diff beat one clever abstraction that is hard to follow. That is a stated trade-off, not an oversight.

4.3. Timer — monotonic host timing

Código 6: include/gol/Timer.hpp — the host stopwatch.

```
1  class Timer {
2  public:
3    Timer() : start_(Clock::now()) {}
4    void reset() { start_ = Clock::now(); }
5    double elapsedMillis() const {
6      return std::chrono::duration<double, std::milli>(Clock::now() - start_).count();
7    }
8  private:
9    using Clock = std::chrono::steady_clock;   // monotonic, not system_clock
10   Clock::time_point start_;
11 };
```

It uses `steady_clock`, not `system_clock`, because it is monotonic: it cannot jump backwards if the wall clock is adjusted mid-run, which would corrupt a timing. This is the CPU backend's kernel-timing source; the GPU backends will time with `cudaEvent_t` / CL events instead.

4.4. ISimEngine and IRenderer — the two strategy interfaces

Código 7: include/gol/ISimEngine.hpp — the engine contract.

```
1  class ISimEngine {
2  public:
```

```
  virtual ~ISimEngine() = default;
  virtual void upload(const Grid& initial) = 0;   // host -> "current" device buffer
  virtual void step() = 0;                        // read current, write next, swap
  virtual void download(Grid& out) = 0;           // device -> host
  virtual double lastKernelMillis() const = 0;    // compute-only time of last step()
  virtual std::string name() const = 0;           // "cpu", "cuda", ...
};
```

The crucial part is the documented **ping-pong contract**: an engine keeps two equally-sized buffers; `step()` reads the "current" buffer, writes the "next" buffer, then swaps them. This guarantees `step()` advances exactly one generation and never aliases its input and output.

Why the two-buffer rule is non-negotiable.

A stencil reads each cell's neighbours. If `step()` wrote new values into the same buffer it is still reading from, cells processed later in the sweep would see *next-generation* neighbours instead of *current-generation* ones, and the result would silently depend on traversal order. The two-buffer invariant prevents that, and it is stated in the interface so every backend obeys it. GPU engines do the swap on the device; only `upload`/`download` cross the bus.

Código 8: include/gol/IRenderer.hpp — the output contract.

```
class IRenderer {
public:
  virtual ~IRenderer() = default;
  virtual void render(const Grid& grid, std::uint64_t generation) = 0;
  virtual bool shouldClose() const { return false; } // interactive renderers override
  virtual bool staysOpen()  const { return false; }  // hold final frame until quit?
};
```

`shouldClose()` defaults to `false`; only an interactive GUI renderer would override it to break the loop early. The key implementation is `NullRenderer` (Section 11), whose entire purpose is to *not exist* in the timed path.

4.5. Config — the run configuration

Código 9: include/gol/Config.hpp — the knobs (abridged).

```
enum class EngineKind { Cpu, Cuda, OpenCL };
enum class RendererKind { Null, Text, Ansi };

struct Config {
  std::size_t rows = 256, cols = 256;
  std::uint64_t generations = 100;
  EngineKind engine = EngineKind::Cpu;
  RendererKind renderer = RendererKind::Null;
```

```
  unsigned threads = 1;       // 1=sequential, N=parallel, 0=all hardware cores
  int blockSize = 256;        // GPU: threads per block (swept in the report)
  bool useShared = false;     // GPU: shared/local-memory tiling
  bool wrap = false;          // false=bounded edges, true=toroidal
  std::uint64_t seed = 1;
  std::optional<std::string> rlePath;
};
Config parse(int argc, char** argv);
```

The GPU-only knobs (`blockSize`, `useShared`) live here and are simply ignored by `CpuEngine`. That is deliberate: one `Config` type, and a single binary whose flag surface is the *union* over all backends, can script the entire benchmark matrix (block sizes $\{32, 64, 128, 256\}$ × shared/no-shared) without recompilation.

5. Configuration parsing — Config.cpp

This translation unit turns `argv` into a fully-resolved `Config`. Three file-private helpers carry the logic.

Código 10: src/core/Config.cpp — value consumption and integer parsing.

```
[[noreturn]] void usageAndExit(int code) { /* prints help */ std::exit(code); }

std::string nextValue(int argc, char** argv, int& i, const char* flag) {
  if (i + 1 >= argc) { std::cerr << "error: " << flag << " requires a value\n";
                       usageAndExit(2); }
  return argv[++i];                    // consume the value AND advance caller's i
}

unsigned long long toULL(const std::string& s, const char* flag) {
  try { return std::stoull(s); }
  catch (const std::exception&) {
    std::cerr << "error: " << flag << " expects an integer, got '" << s << "'\n";
    usageAndExit(2);
  }
}
```

`usageAndExit` is marked `[[noreturn]]`: it never returns (it ends in `std::exit`), which lets call sites fall through after calling it without the compiler complaining about a missing return value.

`nextValue` is the mechanism that makes the parse loop correct. It takes the loop index `i` **by reference**; line 36 checks a token follows the flag, and line 40 *pre-increments* `i` — this both returns the value token and advances the caller's index past it, so the main `for` loop does not re-examine a flag's argument as if it were another flag. Remove the `&` on `i` and the parser would try to interpret every value as a flag.

toULL centralises integer parsing: std::stoull throws std::invalid_argument on non-numeric input and std::out_of_range on overflow; both are caught to print a friendly message and exit, instead of an uncaught exception aborting the program.

Código 11: src/core/Config.cpp — the dispatch loop (representative branches).

```
Config parse(int argc, char** argv) {
  Config cfg;                              // start from defaults
  for (int i = 1; i < argc; ++i) {            // skip argv[0]
    const std::string arg = argv[i];
    if (arg == "-h" || arg == "--help") usageAndExit(0);
    else if (arg == "-r" || arg == "--rows")
      cfg.rows = (std::size_t)toULL(nextValue(argc, argv, i, "--rows"), "--rows");
    else if (arg == "--wrap")  cfg.wrap = true;          // boolean flag, no value
    else if (arg == "--rle")   cfg.rlePath = nextValue(argc, argv, i, "--rle");
    else if (arg == "--engine") {
      const std::string v = nextValue(argc, argv, i, "--engine");
      if (v == "cpu") cfg.engine = EngineKind::Cpu;
      else if (v == "cuda") cfg.engine = EngineKind::Cuda;
      else if (v == "opencl") cfg.engine = EngineKind::OpenCL;
      else { std::cerr << "error: unknown engine '" << v << "'\n"; usageAndExit(2); }
    }
    // ... --cols --gens --threads --seed --renderer --block --shared ...
    else { std::cerr << "error: unknown option '" << arg << "'\n"; usageAndExit(2); }
  }
  return cfg;
}
```

Each value-taking branch uses the idiom toULL(nextValue(...)), which consumes and parses in one expression. –wrap, –shared are value-less booleans. –block/–shared are accepted even in a CPU-only build — that is the union-flag-surface point in code.

What can go wrong.

Missing value, non-numeric value, unknown flag, and unknown engine/renderer name are all caught. One unguarded edge: a negative number passed to –rows is accepted by stoull as a huge unsigned value (a known stoull quirk); it would surface immediately as a failed allocation.

6. The application loop and the measurement invariant — main.cpp

main.cpp owns the loop, wires the chosen engine and renderer, seeds the board, runs the generations, and prints the metric. It is where requirement 2 from Section 2 — keeping I/O out of the number — is enforced.

Código 12: src/core/main.cpp — the factory and the seeder.

```
std::unique_ptr<IRenderer> makeRenderer(RendererKind kind) {
  switch (kind) {
    case RendererKind::Text: return std::make_unique<TextRenderer>();
    case RendererKind::Ansi: return std::make_unique<AnsiRenderer>();
    case RendererKind::Null:
    default: return std::make_unique<NullRenderer>();   // safe default = benchmark-safe
  }
}

void seed(Grid& grid, const Config& cfg) {
  if (cfg.rlePath) {
    Pattern p = RleLoader::load(*cfg.rlePath);
    const std::size_t ox = cfg.cols > p.width  ? (cfg.cols - p.width)  / 2 : 0;
    const std::size_t oy = cfg.rows > p.height ? (cfg.rows - p.height) / 2 : 0;
    p.applyTo(grid, ox, oy);                            // centre the pattern
  } else {
    grid.randomize(cfg.seed);                           // deterministic fill
  }
}
```

The factory's `default` falling to `NullRenderer` is a safety choice: any unexpected enum value degrades to the benchmark-safe no-op renderer rather than crashing. In `seed`, the ternaries on the origins are *not* cosmetic: `cols`, `width` are unsigned, so if the pattern were wider than the grid, `cfg.cols - p.width` would underflow to a gigantic value and `applyTo` would clip the whole pattern off-grid. The guard makes the origin 0 in that case so at least the top-left lands.

Código 13: src/core/main.cpp — the loop and the metric.

```
engine.upload(grid);

// Generation 0 is the seed itself: render it before the first step so frames
// are labelled seed, step 1, step 2, ... The NullRenderer path skips this.
if (rendering) renderer->render(grid, 0);

Timer wall;
double kernelMs = 0.0;
for (std::uint64_t gen = 0; gen < cfg.generations; ++gen) {
  engine.step();
  kernelMs += engine.lastKernelMillis();
  if (rendering) {                    // <-- the whole I/O path is gated here
    engine.download(grid);
    renderer->render(grid, gen + 1);     // state after gen+1 steps
    if (renderer->shouldClose()) break;
  }
}
```

```
18 const double wallMs = wall.elapsedMillis();
19
20 // Interactive viewers hold on the final frame until the user quits; staysOpen()
21 // is false for Null/Text, so this is skipped (and never hangs a pipe).
22 if (rendering && renderer->staysOpen())
23   while (!renderer->shouldClose()) renderer->render(grid, cfg.generations);
24 renderer.reset(); // restore the terminal before printing the summary
25
26 const double cells = (double)cfg.rows * (double)cfg.cols * (double)cfg.generations;
27 auto mcellsPerSec = [cells](double ms) {
28   return ms > 0.0 ? cells / (ms / 1000.0) / 1e6 : 0.0;
29 };
```

The measurement invariant, in code.

The `rendering` bool (set to `cfg.renderer != RendererKind::Null`) gates the entire `download + render` block. Under `NullRenderer` the loop body is *only* `step()` plus the time accumulation: no transfers, no formatting. That is why the rule "always benchmark against `NullRenderer`" holds — it is structurally enforced, not a convention you have to remember.

Two clocks.

`kernelMs` accumulates each step's backend-native compute time (`lastKernelMillis`); `wall` measures the whole loop. Reporting both separately lets the report distinguish "the kernel got faster" from "we are transfer-bound".

Two subtleties.

The seed is generation 0 and is rendered *before* the first `step()` so frame labels are honest (this is the fix in commit `0f87fd9`); the `(double)` casts on the `cells` product avoid 32-bit overflow at large grids (e.g. $4096^2 \times 1000 \approx 6.9 \times 10^{10}$ overflows 32-bit but is exact in `double`), and the `ms > 0.0` guard avoids a divide-by-zero yielding `inf`/NaN.

The whole body runs inside a `try/catch` so that, e.g., a missing RLE file (which throws in the loader) is reported cleanly on `stderr` with exit code 1. A guard before the loop rejects `–engine cuda|opencl` because those backends are not built yet.

7. The CPU engine — CpuEngine.cpp

This is the most subtle file in the project, and the most important: it is the correctness oracle for every future backend, and it implements the sequential baseline and the data-parallel version *in one code path*. The reason one path can serve both is that the Game of Life step has **no intra-generation data dependencies**: every cell reads only the current buffer and writes only the next, so parallelising is *purely* a matter of splitting the rows across threads. The per-cell math is identical; only the row range differs.

7.1. Data model and invariants

Código 14: include/gol/engines/CpuEngine.hpp — the state (abridged).

```
class CpuEngine final : public ISimEngine {
  // ... interface methods ...
  unsigned threads_;  bool wrap_;
  std::size_t rows_ = 0, cols_ = 0;
  std::vector<unsigned char> cur_;   // current generation (read by step)
  std::vector<unsigned char> nxt_;   // scratch for the next generation
  double lastMs_ = 0.0;
  // Persistent worker pool (only populated when threads_ >= 2):
  std::vector<std::thread> workers_;
  std::unique_ptr<std::barrier<>> barrier_;
  const unsigned char* src_ = nullptr;   // buffers published to workers each step
  unsigned char* dst_ = nullptr;
  std::atomic<bool> stop_{false};
};
```

- `cur_` / `nxt_`: the two ping-pong buffers. Invariant: at the start of every `step()`, `cur_` holds the current generation; at the end, after a swap, it holds the new one.
- `workers_`: a *persistent* thread pool, created only when `threads_ >= 2`. Persistent so per-generation thread *creation* never enters the timed region.
- `barrier_`: a `std::barrier` with exactly `threads_` participants (main thread + `threads_`-1 workers).
- `src_` / `dst_`: raw pointers "published" to the workers each step so they know which buffers to read/write.
- `stop_`: an atomic flag used only at destruction.

The class is non-copyable and non-movable: it owns `std::thread`s and a `std::barrier`, neither of which has sensible copy/move semantics.

7.2. Thread resolution, construction, and shutdown

Código 15: src/engines/cpu/CpuEngine.cpp — lifecycle.

```
unsigned CpuEngine::resolveThreads(unsigned requested) {
  if (requested != 0) return requested;
  unsigned hw = std::thread::hardware_concurrency();
  return hw == 0 ? 1u : hw;          // hardware_concurrency may report 0
}
```

```
6
7  CpuEngine::CpuEngine(unsigned threads, bool wrap)
8      : threads_(resolveThreads(threads)), wrap_(wrap) {
9    if (threads_ >= 2) startPool();      // sequential path touches no threads at all
10 }
11
12 CpuEngine::~CpuEngine() {
13   if (!workers_.empty()) {
14     stop_.store(true, std::memory_order_relaxed);
15     barrier_->arrive_and_wait();       // release parked workers ONE last time
16     for (auto& w : workers_) w.join();
17   }
18 }
```

`resolveThreads` maps the request to a concrete count. `0` means "all cores", but `hardware_concurrency()` is allowed to return 0 when it cannot detect the count, so it falls back to 1. This guarantees `threads_ >= 1` always, which everything else relies on (e.g. `rangeFor` divides by it).

The destructor is the trickiest correctness point in the file. At rest, every worker is parked at the phase-1 barrier inside `workerLoop`. To shut down: set `stop_` (relaxed ordering suffices — the barrier provides the happens-before edge that publishes the flag), then `arrive_and_wait()` *once*. That single arrival completes phase 1 and releases every worker, which then sees `stop_` and **returns without arriving at the phase-2 barrier**. That asymmetry is essential: if the workers hit phase 2 on shutdown, the main thread (which is not waiting on phase 2 here) would leave them blocked and `join()` would deadlock.

7.3. The worker pool and the two-phase handshake

Código 16: src/engines/cpu/CpuEngine.cpp — pool startup and worker loop.

```
1  void CpuEngine::startPool() {
2    // participants = main thread + (threads_ - 1) workers = threads_
3    barrier_ = std::make_unique<std::barrier<>>((std::ptrdiff_t)threads_);
4    workers_.reserve(threads_ - 1);
5    for (unsigned id = 1; id < threads_; ++id)       // id 0 is the main thread
6      workers_.emplace_back([this, id] { workerLoop(id); });
7  }
8
9  void CpuEngine::workerLoop(unsigned id) {
10   while (true) {
11     barrier_->arrive_and_wait();                  // phase 1: wait for work
12     if (stop_.load(std::memory_order_relaxed)) return;
13     auto [yBegin, yEnd] = rangeFor(id);
14     computeRows(src_, dst_, yBegin, yEnd);
```

```
15    barrier_->arrive_and_wait();                // phase 2: signal completion
16  }
17 }
```

Worker ids run $1 \ldots \text{threads\_} - 1$; **id 0 is the main thread**. The main thread doing real work (partition 0) rather than just coordinating is a deliberate efficiency choice: with N cores you get N workers, not $N - 1$ plus an idle coordinator.

The worker is a two-phase handshake. **Phase 1** ("go") blocks until `step()` or the destructor arrives. After release the worker checks `stop_` — the only way the loop ever ends. Otherwise it computes its row slice and signals **phase 2** ("done"), then loops back to park at phase 1. The two distinct barriers per cycle separate "all threads have started this generation" from "all threads have finished it".

Why the lock-free pointer publish is safe.

`step()` writes `src_`/`dst_` *before* arriving at phase 1; the worker reads them *after* leaving phase 1. The barrier establishes a happens-before relationship, so the writes are visible without any lock or atomic on the pointers themselves.

7.4. Row partitioning — rangeFor

Código 17: src/engines/cpu/CpuEngine.cpp — the half-open row range for a partition.

```
1 std::pair<std::size_t, std::size_t> CpuEngine::rangeFor(unsigned id) const {
2   const std::size_t base = rows_ / threads_;
3   const std::size_t rem  = rows_ % threads_;
4   const std::size_t begin = id * base + std::min<std::size_t>(id, rem);
5   const std::size_t count = base + (id < rem ? 1u : 0u);
6   return {begin, begin + count};
7 }
```

The first `rem` partitions each get one extra row. Worked example with `rows_ = 17`, `threads_ = 4` (base = 4, rem = 1):

Tabla 2: `rangeFor` over 17 rows, 4 threads. Rows are contiguous, with no gaps or overlaps, and sum to 17.

id	base	extra	begin	range
0	4	1	0	$[0, 5)$
1	4	0	5	$[5, 9)$
2	4	0	9	$[9, 13)$
3	4	0	13	$[13, 17)$

The `min(id, rem)` term is the clever bit: partitions before `rem` each contributed one

extra row to the running offset, so partition `id`'s start is shifted by `min(id, rem)`. Note that with `threads_ == 1` this returns $[0, \text{rows\_})$ — the full grid — so the sequential sweep is the degenerate single-partition case. The `RemainderRowsHandled` test (Section 14) targets exactly this logic.

7.5. Neighbour counting and the stencil

Código 18: src/engines/cpu/CpuEngine.cpp — both edge modes (condensed).

```
int CpuEngine::countNeighbors(const unsigned char* g, std::size_t x,
                              std::size_t y) const {
  int n = 0;
  if (wrap_) {                              // toroidal: indices wrap
    const long cols = (long)cols_, rows = (long)rows_;
    for (long dy = -1; dy <= 1; ++dy)
      for (long dx = -1; dx <= 1; ++dx) {
        if (dx == 0 && dy == 0) continue;       // a cell is not its own neighbour
        const std::size_t nx = ((long)x + dx + cols) % cols;   // +size before %
        const std::size_t ny = ((long)y + dy + rows) % rows;
        n += g[ny * cols_ + nx];
      }
  } else {                                  // bounded: OOB counts as dead
    for (long dy = -1; dy <= 1; ++dy)
      for (long dx = -1; dx <= 1; ++dx) {
        if (dx == 0 && dy == 0) continue;
        const long nx = (long)x + dx, ny = (long)y + dy;
        if (nx < 0 || ny < 0 || nx >= (long)cols_ || ny >= (long)rows_) continue;
        n += g[(std::size_t)ny * cols_ + (std::size_t)nx];
      }
  }
  return n;
}

void CpuEngine::computeRows(const unsigned char* src, unsigned char* dst,
                            std::size_t yBegin, std::size_t yEnd) const {
  for (std::size_t y = yBegin; y < yEnd; ++y) {
    const std::size_t row = y * cols_;          // hoisted out of inner loop
    for (std::size_t x = 0; x < cols_; ++x)
      dst[row + x] = nextState(src[row + x], countNeighbors(src, x, y));
  }
}
```

Cells are 0/1, so *summing* neighbour values *is* counting the live ones — no comparison needed. In the toroidal branch the `+ size` before `%` is necessary because C++'s `%` on a negative operand yields a negative result; adding the size first guarantees a non-negative

dividend, so -1 mod cols becomes cols $-$ 1 (wrap to the far edge). In the bounded branch, out-of-range neighbours are simply skipped, i.e. counted as dead — this is the **default** edge mode. Both modes must match every backend bit-for-bit, which is why both are tested.

`computeRows` is the single implementation of the stencil, called by *both* the sequential path (full range) and every worker (its slice). One implementation of the math is the source of the sequential/parallel equivalence. Hoisting `row = y*cols_` out of the inner loop saves a multiply per cell.

7.6. upload, step, download

Código 19: src/engines/cpu/CpuEngine.cpp — the per-generation core.

```
1  void CpuEngine::upload(const Grid& initial) {
2    rows_ = initial.rows();  cols_ = initial.cols();
3    cur_.assign(initial.data(), initial.data() + initial.size());
4    nxt_.assign(cur_.size(), 0u);
5  }
6
7  void CpuEngine::step() {
8    Timer t;                                  // CPU's lastKernelMillis source
9    if (workers_.empty()) {
10     computeRows(cur_.data(), nxt_.data(), 0, rows_);  // SEQUENTIAL: one full sweep
11   } else {
12     src_ = cur_.data();  dst_ = nxt_.data();        // publish buffers to workers
13     barrier_->arrive_and_wait();                  // phase 1: release workers
14     auto [yBegin, yEnd] = rangeFor(0);
15     computeRows(src_, dst_, yBegin, yEnd);          // main thread = partition 0
16     barrier_->arrive_and_wait();                  // phase 2: wait for all
17   }
18   std::swap(cur_, nxt_);                         // the ping-pong
19   lastMs_ = t.elapsedMillis();
20 }
21
22 void CpuEngine::download(Grid& out) {
23   std::copy(cur_.begin(), cur_.end(), out.data());   // host copy; device->host on GPU
24 }
```

`upload` records the dimensions, copies the seed into `cur_`, and sizes `nxt_` (its initial contents do not matter — `step` overwrites every cell).

`step` is timed by the chrono `Timer`; this is the CPU's `lastKernelMillis()` source. In the **sequential** branch it is one `computeRows` over $[0, \text{rows\_})$ with zero synchronisation. In the **parallel** branch the main thread mirrors the worker handshake from its own side: publish the buffer pointers, phase-1 arrival releases the workers, the main thread computes partition 0 itself instead of idling, and phase-2 arrival blocks until every worker has finished. The symmetry is the point: `step` runs the same phase1→compute→phase2 dance that each

worker runs, which is why they stay in lockstep generation after generation. The std::swap is the ping-pong: nxt_ becomes the new cur_.

download copies the current buffer into the caller's grid. On the CPU this is the "no-op cost" the interface mentions — a host copy; on the CUDA backend this same method is the device→host transfer (cudaMemcpy D2H), which is precisely why it is excluded from the benchmark.

What would break.

Remove the stop_ check and the destructor deadlocks. Swap the order of "publish src_/dst_" and "phase-1 arrive" and workers could read stale pointers. Give nextState any per-cell state and the sequential/parallel equivalence collapses — it is stateless on purpose.

8. The CUDA engine — CudaEngine.cu and kernel.cu

The GPU backend is the first that crosses the host/device boundary, yet it reuses everything the core already fixed: the same flat Grid bytes, the same nextState rule (now compiled __host__ __device__ under nvcc), and the same edge handling. That reuse is exactly why it can be checked bit-for-bit against the CPU oracle (Section 14).

8.1. Engine: device buffers, ping-pong, event timing

CudaEngine owns two device buffers (dCur_, dNxt_) and does the ping-pong on the device; the host Grid is touched only by upload (H2D cudaMemcpy) and download (D2H). The header is deliberately CUDA-free (handles stored as void* / unsigned char* and cast in the .cu), so main.cpp compiles with the host compiler and only the .cu files go through nvcc.

Código 20: src/engines/cuda/CudaEngine.cu — one generation, kernel-only timing.

```
void CudaEngine::step() {
  cudaEventRecord(start);
  launchLifeStep(dCur_, dNxt_, rows_, cols_, wrap_, blockSize_, useShared_);
  cudaEventRecord(stop);
  cudaEventSynchronize(stop);
  cudaEventElapsedTime(&ms, start, stop);  // kernel time only -- excludes H2D/D2H
  lastMs_ = ms;
  std::swap(dCur_, dNxt_);                 // ping-pong on the device
}
```

Bracketing the launch with cudaEvent_t makes lastKernelMillis() report kernel-only time, excluding the transfers — the GPU analogue of the CPU's chrono timing, and what makes the cells/sec numbers comparable across backends (Section 15.1).

8.2. Kernels: one thread per cell, 32-wide tiles

kernel.cu holds two device kernels and a host launcher. The launcher maps the block size onto a 2-D tile and selects the variant:

Código 21: src/engines/cuda/kernel.cu — launcher (tile mapping + kernel choice).

```
const int bx = 32;                    // 32-wide rows -> coalesced global loads
const int by = (blockSize + bx - 1) / bx;   // 32/64/128/256 -> by = 1/2/4/8
dim3 block(bx, by);
dim3 grid((cols + bx - 1) / bx, (rows + by - 1) / by);
if (useShared)
  lifeShared<<<grid, block, (bx + 2) * (by + 2)>>>(src, dst, rows, cols, wrap);
else
  lifeGlobal<<<grid, block>>>(src, dst, rows, cols, wrap);
```

The 32-wide tile is the key performance choice: consecutive threads in a warp read consecutive 1-byte cells of a row, so global loads coalesce — the right default for a memory-bound stencil. lifeGlobal reads the eight neighbours straight from global memory. lifeShared (selected by −shared) first stages a (bx+2) x (by+2) tile plus a one-cell halo into shared memory cooperatively, applying the *same* edge rule during the halo load (bounded → 0, toroidal → wrap), then reads neighbours from the tile. Both produce byte-identical results to the CPU oracle; the report's job is to measure whether the staging pays off (Section 15 predicts it largely will not for a 1-byte stencil).

8.3. Three coordinate spaces: thread, board, and memory

The single most common source of confusion in the CUDA backend is collapsing three distinct coordinate systems into one. The kernel juggles all three, and keeping them separate makes the index arithmetic obvious (Figure 2):

- **Execution space** — how the *threads* are organised: the dim3 grid/block the launcher picks, addressed by the 2-D blockIdx and threadIdx. This says nothing about memory; it is only how work is handed out.
- **Board space** — the rows × cols logical Game-of-Life grid, addressed by (row, col). This is where "above", "below", "left" and "right" mean something.
- **Memory space** — the flat, row-major 1-D Grid array, addressed by a single offset.

The Grid is *always* 1-D; the 2-D thread indexing exists only to assign one thread per cell and to make neighbour arithmetic cheap. Two maps connect the spaces, and each is a single line of kernel code. The launcher's thread indices become board coordinates (kernel.cu:30–31): col = blockIdx.x*32 + threadIdx.x and row = blockIdx.y*by + threadIdx.y; and

the instant the kernel touches memory it flattens back to 1-D (kernel.cu:46,49): `offset = row*cols + col`. That single `*cols` is the hinge that ties the three spaces together.

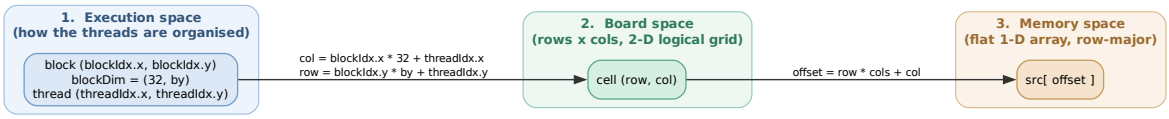

Figura 2: The three coordinate spaces and the two maps between them. Threads are indexed in 2-D (execution space) only for convenience; the board is the 2-D logical grid; storage is a flat 1-D array. `col`/`row` come from the thread indices, and `offset = row*cols + col` flattens a board cell to its array slot.

A block is a rectangle on the board, not a run in memory.

The block shape $32 \times$ `by` is $32 \times$ `by` adjacent cells *in board space* — a small rectangle — which is *not* $32 \times$ `by` adjacent cells in memory. Each of the `by` rows of the block is a run of 32 contiguous bytes, and consecutive rows sit `cols` apart, because that is what "the row below" means in row-major storage. Figure 3 makes this concrete on a scaled-down 8×6 board with a 4×2 block: the block's eight cells are two contiguous 4-runs separated by a `cols - bx` gap. A *warp* — the 32 threads (here drawn as 4) sharing one `threadIdx.y` — is exactly one such contiguous run, which is why those loads coalesce into a single memory transaction and why `bx` is pinned to 32. The cell directly "below" a thread is `+cols` away in memory, never `+bx`.

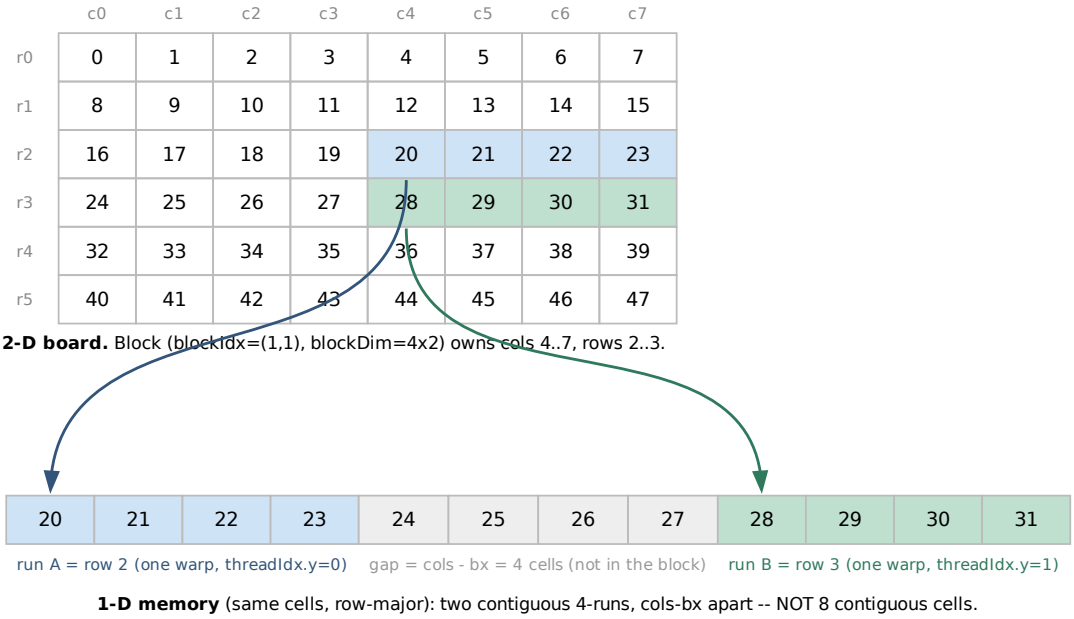

Figura 3: One block maps to a rectangle on the board (top) but to two *discontiguous* runs in memory (bottom), `cols - bx` apart. Scaled down (`cols=8`, 4×2 block, "warp" = 4) to fit; real code uses a 32-wide warp. Cell numbers are the 1-D offset `row*cols + col`. Each warp (fixed `threadIdx.y`) is one contiguous run, so its loads coalesce.

Neighbours are reconstructed, not stored adjacently.

Because storage is 1-D, the nine cells of the stencil are not contiguous either. The kernel reaches each neighbour by recomputing its offset (Figure 4): the two horizontal neighbours are `idx ± 1`, but the two vertical neighbours are `idx ± cols` — a whole row away — and the diagonals combine both. This is exactly the `nr*cols + nc` expression in `lifeGlobal`'s neighbour loop (`kernel.cu:46`): the 2-D adjacency of the board is rebuilt arithmetically from the 1-D layout on every access, so the same flat `Grid` serves every backend unchanged.

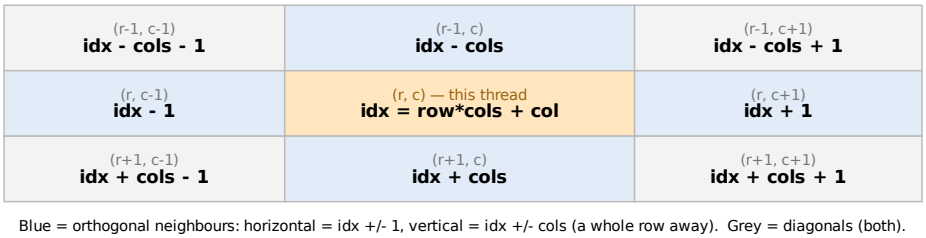

Figura 4: The 9-point stencil expressed in memory offsets. Horizontal neighbours are `idx` ± `1`; vertical neighbours are `idx` ± `cols` (a whole row away); diagonals combine both. The kernel computes these as `nr*cols + nc`.

8.4. The two swept options

Block size (`-block`) and shared memory (`-shared`) are the two assignment options chosen (#1 and #3); both are runtime-selectable, so one binary drives the whole sweep. The CUDA target is opt-in (`-DBUILD_CUDA=ON`), built for `sm_75` (the GTX 1660 Ti) with `-lineinfo` so Nsight Compute can attribute time to source lines.

9. The OpenCL engine — OpenCLEngine.cpp and kernel.cl

`OpenCLEngine` is the CUDA engine's twin: one work-item per cell over a 2D NDRange, two device buffers, a device-side ping-pong, and the same two runtime options (`-block` sizes the work-group, `-shared` selects the local-memory kernel). The design rationale is identical to Section 8; this section covers only what differs.

The kernel is read at runtime, not compiled with the host.

CUDA's kernels are compiled ahead of time by `nvcc`; OpenCL's are not. The engine reads `src/engines/opencl/kernel.cl` from disk at start-up and hands the source to `clCreateProgramWithSource` + `clBuildProgram`, which the NVIDIA driver JIT-compiles for the present device. This is why `kernel.cl` must stay diff-able against `LifeRules.hpp`: it is plain C and cannot `#include` the C++ header, so it carries a hand-mirrored copy of `nextState` and the edge rule (Section 4.2). Its two kernels `life_global` and `life_local` mirror CUDA's `lifeGlobal` / `lifeShared`; `life_local` stages a $(bx{+}2) \times (by{+}2)$ tile plus halo into a `__local` buffer and synchronises with `barrier(CLK_LOCAL_MEM_FENCE)`.

Timing uses command-queue events.

The queue is created with CL_QUEUE_PROFILING_ENABLE; each step enqueues the kernel via clEnqueueNDRangeKernel, waits on its event, and subtracts CL_PROFILING_COMMA from _END to get the device execution time — the OpenCL analogue of CUDA's cudaEvent_t bracket, and likewise excluding the host ↔ device copies.

Same backend as CUDA on NVIDIA.

On an NVIDIA box the OpenCL platform is "NVIDIA CUDA", so the JIT lowers kernel.cl through the same PTX/SASS backend as the CUDA path. The two engines therefore perform within measurement noise of each other: the ∼5% gap that shows up between their peaks is smaller than the GPU's ∼25–30% run-to-run boost-clock variance (Section 15), so neither is meaningfully faster. The engine is opt-in (-DBUILD_OPENCL=ON) and verified bit-for-bit against the CpuEngine oracle, both at runtime (–engine opencl –verify) and in gol_opencl_tests.

10. The pattern pipeline — RleLoader.cpp and Pattern.cpp

A Pattern is a *sparse* seed: a bounding box plus the list of live (x,y) offsets. Sparse because that is the natural output of RLE decoding and it decouples a pattern from any particular grid size.

Código 22: src/patterns/RleLoader.cpp — the decode loop (core).

```
std::size_t x = 0, y = 0, count = 0;
bool haveCount = false;
for (const char ch : body) {                    // body = payload, newlines dropped
  if (std::isspace((unsigned char)ch)) continue;
  if (std::isdigit((unsigned char)ch)) {
    count = count * 10 + (std::size_t)(ch - '0');  // build multi-digit run lengths
    haveCount = true; continue;
  }
  const std::size_t n = haveCount ? count : 1;     // count defaults to 1
  bool done = false;
  switch (ch) {
    case 'b': x += n; break;                              // dead run: advance x
    case 'o': for (std::size_t i=0;i<n;++i) pat.liveCells.emplace_back(x++, y); break;
    case '$': y += n; x = 0; break;                        // end of row(s)
    case '!': done = true; break;                         // end of pattern
    default: break;                                      // ignore unexpected
  }
  count = 0; haveCount = false;
  if (done) break;
}
```

The RLE grammar is <count><tag> with the count defaulting to 1. Digits accumulate into count; on a tag, n is the accumulated count or 1, and the four tags drive a cursor: b skips dead cells, o emits n live cells (advancing x), $ ends rows (and n$ skips n blank rows at once — how RLE compresses empty space), ! terminates. The payload is read into one body string with newlines removed first, because RLE tokens can wrap across lines and cannot be decoded line-by-line.

A concrete trace.

The birth_on_six.rle payload 3ooboo! decodes as:

Código 23: Decoding 3ooboo! – six live cells around a DEAD centre (1,1).

```
3o  -> (0,0) (1,0) (2,0)     x=3 y=0
$   -> y=1 x=0
o   -> (0,1)               x=1
b   -> x=2                  (skips (1,1): the centre stays DEAD)
o   -> (2,1)               x=3
$   -> y=2 x=0
o   -> (0,2)               x=1
!   -> stop
```

That leaves a dead centre $(1, 1)$ surrounded by exactly six live neighbours — which under the variant rule must be *born*. This fixture is the payload behind the "birth on 6" correctness test (Section 14).

Two robustness details round out the loader: the header parser (x = 3, y = 3, rule = B36/S23) *ignores* the rule= field — the simulation rule lives in code, not data, so a stray fixture cannot silently change the simulation — and if the header omits dimensions, the loader infers the bounding box from $\max(x)+1$, $\max(y)+1$ over the decoded cells. Opening a missing file throws std::runtime_error.

Código 24: src/patterns/Pattern.cpp — stamping, with clipping.

```
void Pattern::applyTo(Grid& grid, std::size_t originX, std::size_t originY) const {
  for (const auto& [dx, dy] : liveCells) {
    const std::size_t x = originX + dx, y = originY + dy;
    if (x < grid.cols() && y < grid.rows())   // clip, don't crash
      grid.at(x, y) = 1u;
  }
}
```

Because x, y are unsigned, the single test x < grid.cols() rejects both "too far right" and any wrap-around at once — no separate >= 0 check is needed. Cells that fall outside are silently dropped, which is the right behaviour for a centering seeder: a pattern slightly larger than the grid still produces a valid, cropped board rather than a crash.

11. Rendering — NullRenderer, TextRenderer, AnsiRenderer

There are three renderers behind IRenderer, none of which is in the benchmark path (the loop only renders when the renderer is not Null). NullRenderer::render is an empty inline body — header-only because there is nothing to define. Its entire purpose is to *not* exist in the timed path (Section 6).

Código 25: src/render/TextRenderer.cpp — one buffer, one write.

```
void TextRenderer::render(const Grid& grid, std::uint64_t generation) {
  const std::size_t glyph = std::max(alive_.size(), dead_.size()); // bytes per cell
  std::string out;
  out.reserve(grid.size() * glyph + grid.rows() + 32);
  out += "generation "; out += std::to_string(generation); out += '\n';
  for (std::size_t y = 0; y < grid.rows(); ++y) {
    for (std::size_t x = 0; x < grid.cols(); ++x)
      out += (grid.at(x, y) ? alive_ : dead_);   // alive_/dead_ are std::string
    out += '\n';
  }
  out += '\n';
  std::fputs(out.c_str(), stdout);              // single write, no per-cell putchar
}
```

The mechanism worth noting is that it builds the entire frame into one std::string (pre-reserved so it never reallocates) and emits it with a single fputs. Per-cell putchar would lock and flush the stream thousands of times per frame. at(x, y) keeps the (column, row) argument order matching Grid's convention even though storage is row-major. A terminal character cell is about twice as tall as it is wide, so to make cells read as squares a live cell defaults to *two* full blocks (U+2588) and a dead cell to two spaces; the glyph fields are std::string (not char) to hold the multi-byte, two-column cell, and alive_ / dead_ share a display width so the grid stays aligned. This renderer is for eyeballing correctness only — never in a benchmark.

11.1. AnsiRenderer — in-place animation, clipped to the viewport

TextRenderer *appends* each frame, so the terminal scrolls and any row wider than the terminal soft-wraps; it is a dump, not an animation. AnsiRenderer instead homes the cursor and overwrites the same region every frame using ANSI/DEC control sequences, producing a stable in-place animation. It is a small-grid visualisation aid; benchmarks still use NullRenderer.

Código 26: src/render/AnsiRenderer.cpp — the per-frame sequence (condensed).

```cpp
// Construction: hide cursor, disable autowrap (wide rows clip, not wrap), clear.
//   "\033[?25l"  "\033[?7l"  "\033[2J"
// Destruction (RAII): restore "\033[?7h" (wrap on) "\033[?25h" (cursor on).
void AnsiRenderer::render(const Grid& grid, std::uint64_t generation) {
  unsigned termRows = 24, termCols = 80;
  terminalSize(termRows, termCols);                    // ioctl(TIOCGWINSZ), 80x24 fallback
  const std::size_t visRows = std::min(grid.rows(), termRows > 1 ? termRows - 1u : 1u);
  const std::size_t cellCols = std::max<std::size_t>(1, displayWidth(alive_)); // cols/cell (2
    ↪ by default)
  const std::size_t visCols = std::min(grid.cols(), termCols / cellCols);      // clip to
    ↪ viewport
  std::string out;
  out += "\033[?2026h";                                 // begin synchronized (atomic) update
  out += "\033[H";                                     // cursor home (overwrite, no flicker)
  out += "generation " + std::to_string(generation);
  if (visRows < grid.rows() || visCols < grid.cols())   // note the clip window
    out += " [" + std::to_string(visCols) + "x" + std::to_string(visRows) +
           " of " + std::to_string(grid.cols()) + "x" + std::to_string(grid.rows()) + "]";
  out += "\033[K\n";
  for (std::size_t y = 0; y < visRows; ++y) {
    for (std::size_t x = 0; x < visCols; ++x) out += (grid.at(x, y) ? alive_ : dead_);
    out += "\033[K\n";                                  // clear to EOL (wipe leftovers)
  }
  out += "\033[J";                                     // clear below (handles a shrunk terminal)
  out += "\033[?2026l";
  std::fputs(out.c_str(), stdout); std::fflush(stdout);
  if (delayMs_ > 0) std::this_thread::sleep_for(std::chrono::milliseconds(delayMs_));
}
```

Big grids: it clips, it does not wrap.

This is the key behavioural difference. TextRenderer relies on the terminal: a 300×300 board in an 80-column terminal wraps every row and scrolls — unreadable but harmless. AnsiRenderer cannot allow wrapping, because a wrapped row would occupy several terminal rows and desynchronise the cursor-home arithmetic that the whole in-place scheme depends on. So it disables autowrap (\033[?7l) and renders only the top-left $\min(\text{rows}, \text{term} - 1) \times \min(\text{cols}, \text{term/cell})$ window, where *cell* is the per-cell display width (2 for the default two-block glyph); it annotates the header, e.g. generation 0 view(0,0) 40x23 of 300x300 (300 columns in an 80-column terminal leaves room for 40 two-column cells). The size is re-queried every frame, so resizing the terminal is handled.

Interactive controls.

Because the window is smaller than the grid, the renderer lets you drive it. When stdin is a TTY it puts the terminal in raw, non-blocking mode (clearing ICANON/ECHO,

VMIN=VTIME=0) and drains queued key presses each frame: the arrow keys shift the (offsetX_, offsetY_) top-left of the viewport (by ≈ one eighth of the view per press, clamped to the grid), space/p toggles pause, and q sets the flag returned by shouldClose() so the main loop stops. Reads are non-blocking, so input never stalls the simulation.

Pause is contained entirely in the renderer: while paused, render() loops on a do/while — still polling input and redrawing the *same* generation (the header shows [PAUSED]) — and returns only once unpaused or quitting, so the application never calls step() meanwhile. The main loop, the engine, and the IRenderer interface are untouched, and the benchmark path (NullRenderer) never pauses. The terminal is restored on destruction *and* from a SIGINT/SIGTERM handler (so Ctrl-C cannot leave it in raw mode); when stdin is not a TTY, the controls are disabled and the top-left window is shown.

Scrollbars.

To show *where* the viewport sits, the renderer draws a scrollbar in each clipped direction: a vertical bar in the rightmost column (a heavy solid thumb, U+2503, over a dotted track, U+250A) and a horizontal bar in a row under the grid (heavy U+2501 over dotted U+2508) — the solid thumb stands out against the dotted rail. Each is shown only when the grid is clipped that way (one row/column is reserved for it). A shared helper maps the visible band onto the bar — the thumb length is the visible fraction (≈ len × visible / total) and its position tracks the offset — so the bar reads like any GUI scrollbar.

Holding after the run.

staysOpen() returns interactive_, so once the generation loop ends the application keeps calling render() on the final board (still panning/pausing) until q. Piped output reports staysOpen() == false, so a non-TTY run exits normally and never hangs.

Robustness details.

A per-frame \033[?2026h/l pair asks the terminal for a *synchronized* (tear-free) update and is silently ignored where unsupported; \033[K wipes any leftover from a previously wider frame and \033[J wipes rows orphaned by a shrunk terminal; the cursor is hidden during animation and restored (with autowrap) by the destructor even on an early break or exception. A small delayMs (default 50 ms) paces the otherwise-uncapped loop so the animation is watchable; it lives only in this renderer, never in the timed path. The size query is POSIX (ioctl(TIOCGWINSZ)); off a TTY it falls back to 80 × 24.

12. The Game of Life viewer — Qt + OpenGL with CUDA interop

The viewer (gol_gui) is the project's answer to Tarea 3 ("Interop – Shaders"): a real-time window that animates the *same* simulation the headless gol benchmarks. Its reason to exist is one idea — display the GPU-computed board *without copying it back to the host.* The compute API (CUDA) and the graphics API (OpenGL) share one device buffer, so the board is born, stepped, and drawn entirely in GPU memory; only keystrokes cross the PCIe bus.

Everything else in this section follows from making that sharing work, and from degrading gracefully to a plain host copy when it cannot.

12.1. Why it is a separate front-end, not an IRenderer

It would be natural to add a `GuiRenderer` alongside `TextRenderer` and `AnsiRenderer`. That does not work, for two structural reasons. First, **Qt owns the loop**: `QApplication::exec()` *is* the event loop and calls back into the widget (via `paintGL` and timers), so the application cannot keep `main.cpp`'s `for (gen...){ step; render }` loop. Second, `IRenderer::render(const Grid&, gen)` takes a *host* `Grid` — exactly the host copy interop exists to avoid. So the viewer is a **peer front-end** on the same core, not a renderer plugged into the old loop: it is its own executable (`gol_gui`), gated on `BUILD_GUI`, and the headless `gol` never links Qt or OpenGL.

12.2. Components and the strategy it composes

Figure 5 shows the pieces. `MainWindow` (a `QMainWindow`) holds the central `GolGlWidget` and a dockable control panel, wired with Qt signals/slots. `GolGlWidget` (a `QOpenGLWidget`) is the heart: it owns an `ISimEngine` (the unchanged CPU or CUDA engine), a `CudaGlInterop` bridge, the host `Grid` used for seeding and for the host-upload path, and a snapshot `initialGrid_` for Reset. The widget thus *composes* the existing strategy interface rather than extending it.

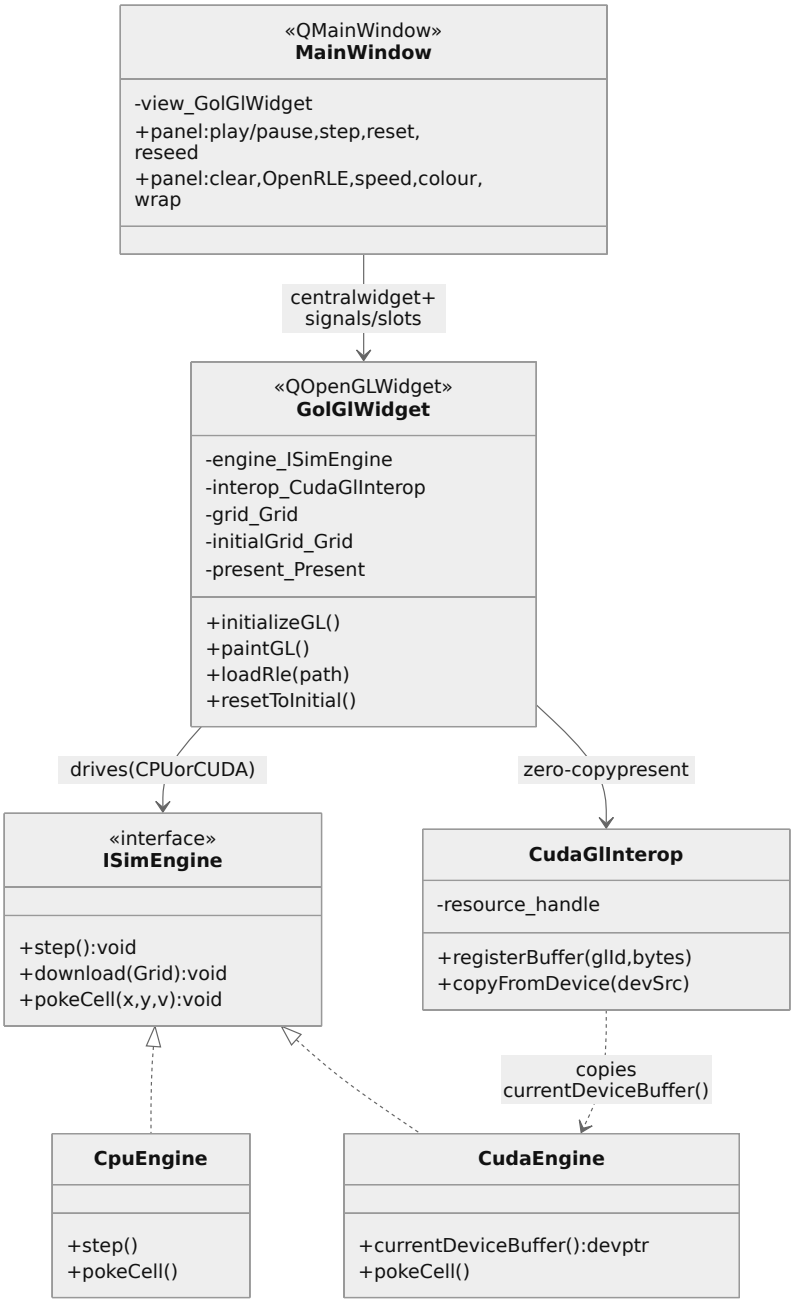

Figura 5: Components of the viewer. `MainWindow` owns `GolGlWidget`; the widget drives an `ISimEngine` (`CpuEngine` or `CudaEngine`) and, for the zero-copy path, a `CudaGlInterop` bridge that copies the engine's current device buffer into a GL buffer. `CpuEngine` and `CudaEngine` both realize `ISimEngine`; only `CudaEngine` exposes `currentDeviceBuffer()`.

Two small, additive changes to the engine layer make this composition possible. `ISimEngine` gains an optional `pokeCell` for live editing, defaulting to a no-op so backends that do not support it (e.g. OpenCL) degrade silently:

Código 27: include/gol/ISimEngine.hpp — optional live editing; default no-op.

```
virtual void pokeCell(std::size_t x, std::size_t y, unsigned char value) {
  (void)x; (void)y; (void)value;   // backends that support it override this
}
```

CpuEngine overrides it with a one-line in-place write to its cur_ buffer; CudaEngine overrides it with a one-byte cudaMemcpy, and also exposes the device pointer the interop bridge needs. The header stays CUDA-free (the pointer is returned as const void*):

Código 28: CudaEngine: a device-pointer accessor and a 1-byte editor.

```
// include/gol/engines/CudaEngine.hpp -- header stays free of CUDA types.
const void* currentDeviceBuffer() const { return dCur_; }
void pokeCell(std::size_t x, std::size_t y, unsigned char value) override;

// src/engines/cuda/CudaEngine.cu
void CudaEngine::pokeCell(std::size_t x, std::size_t y, unsigned char value) {
  if (!dCur_ || x >= cols_ || y >= rows_) return;
  cudaMemcpy(dCur_ + y * cols_ + x, &value, 1, cudaMemcpyHostToDevice);
}
```

12.3. The interop bridge — CudaGlInterop

The bridge is the only place that touches both APIs, so it is isolated in a .cu (compiled by nvcc, which sees cuda_gl_interop.h) behind a CUDA-free header (so the Qt widget compiles with the host compiler). The contract is the classic interop dance: **register once**, then **per frame** map → write → unmap. Registering tells the driver that a GL buffer and a CUDA resource are the same memory; mapping hands ownership to CUDA so neither API touches it mid-operation.

Código 29: src/render/CudaGlInterop.cu — register once, then copy each frame.

```
void CudaGlInterop::registerBuffer(unsigned int glBufferId, std::size_t bytes) {
  // WriteDiscard: CUDA overwrites the whole buffer each frame, so the driver
  // need not preserve its previous contents.
  cudaGraphicsGLRegisterBuffer(&res, glBufferId, cudaGraphicsRegisterFlagsWriteDiscard);
  resource_ = res; bytes_ = bytes;
}

void CudaGlInterop::copyFromDevice(const void* deviceSrc) {
  cudaGraphicsMapResources(1, &res, 0);                 // GL -> CUDA
  cudaGraphicsResourceGetMappedPointer(&dPtr, &mapped, res);
  cudaMemcpy(dPtr, deviceSrc, bytes_, cudaMemcpyDeviceToDevice); // stays on the GPU
  cudaGraphicsUnmapResources(1, &res, 0);               // CUDA -> GL
```

```
}
```

The decisive line is the `cudaMemcpyDeviceToDevice`: `deviceSrc` is the engine's `dCur_` and `dPtr` is the GL buffer's memory, both on the same GPU, so the copy never leaves the device. This is what "zero-copy" means here — zero copies across PCIe, not zero copies at all (the on-GPU D2D copy runs at hundreds of GB/s). Keeping the engine's ping-pong intact and copying *out* of it, rather than letting `step` write straight into the GL buffer, is deliberate: the engine's contract (Section 4.4) is untouched, and the viewer only ever *reads* the current buffer.

12.4. Widget lifecycle — initializeGL

`initializeGL` runs once, with the GL context current (the only safe place to register a GL buffer with CUDA). It compiles the shaders, makes the GL objects the board needs — a Pixel Buffer Object (PBO) that CUDA writes and GL reads, and a single-channel unsigned-integer texture (`GL_R8UI`, one texel per cell) — then builds the engine and tries to register the PBO.

Código 30: src/gui/GolGlWidget.cpp — the PBO and the integer board texture.

```
glGenBuffers(1, &pbo_);
glBindBuffer(GL_PIXEL_UNPACK_BUFFER, pbo_);
glBufferData(GL_PIXEL_UNPACK_BUFFER, bytes, nullptr, GL_DYNAMIC_COPY);

glGenTextures(1, &tex_);
glBindTexture(GL_TEXTURE_2D, tex_);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST); //
    ↪ integer tex: no filtering
glTexImage2D(GL_TEXTURE_2D, 0, GL_R8UI, cols, rows, 0,
             GL_RED_INTEGER, GL_UNSIGNED_BYTE, nullptr);
```

A `GL_R8UI` texture is the right type for a board: it stores the cell byte exactly and is read in the shader with an integer sampler (`usampler2D`) and `texelFetch`, with no filtering — integer textures cannot be linearly filtered, and we want crisp cells anyway. The presentation path is then chosen: default to host-upload (works on any context), and attempt interop only for a `CudaEngine`, falling back rather than failing if the GL context is on a different GPU than CUDA:

Código 31: Choosing the presentation path – interop with a graceful fallback.

```
present_ = Present::HostUpload;            // works on any GL context
if (cudaEngine_) {                       // a CUDA engine? try zero-copy
  try {
    interop_.registerBuffer(pbo_, bytes);
    present_ = Present::Interop;
```

```
  } catch (const std::exception& e) {        // e.g. GL on the Intel iGPU
    // log "interop unavailable on <GL_RENDERER>; falling back to host-upload"
    present_ = Present::HostUpload;
  }
}
```

12.5. Two presentation paths — zero-copy vs host-upload

Figure 6 contrasts the two paths the previous snippet selects between. In **Interop** mode the bridge copies `dCur_` into the PBO (device-to-device), and `glTexSubImage2D` streams the PBO into the texture — both on the GPU. In **HostUpload** mode the engine's `download(grid_)` pulls the board to host RAM (a D2H `cudaMemcpy` for CUDA, a plain copy for the CPU engine), and `glTexSubImage2D` uploads it from client memory — two PCIe crossings. The CPU engine has no device buffer, so it always uses the host-upload path; the CUDA engine uses interop when the GL context is on the NVIDIA GPU and host-upload otherwise.

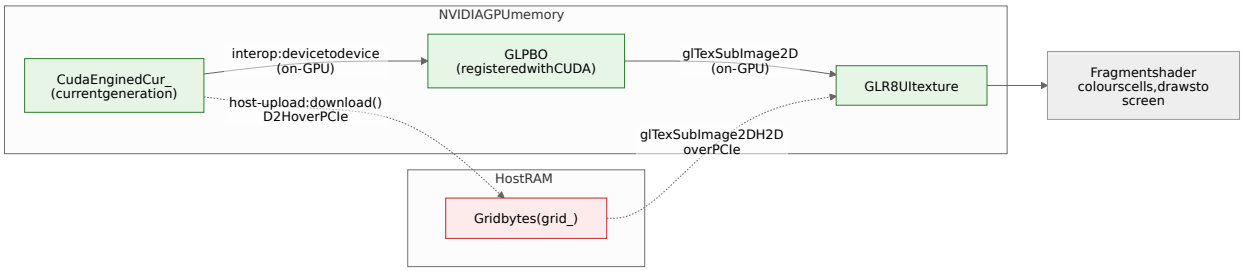

Figura 6: The two presentation paths (CUDA engine). Solid (green) edges stay in GPU memory — the zero-copy interop path, `dCur_` → PBO → texture. Dashed (red) edges cross PCIe — the host-upload fallback, `dCur_` → host `grid_` → texture. The CPU engine always takes the dashed path (its board already lives in host RAM).

12.6. The frame: present, then step

`paintGL` draws one frame. The order matters: it **presents the current board first, then advances**. The obvious order (step, then present) has a subtle bug for interactive editing — a cell painted between frames would be consumed by `step` before it was ever drawn, so a lone live cell (which dies immediately under `B36/S23`, needing two or three neighbours) would seem to vanish while still perturbing nearby patterns. Presenting first means every edit is visible for the frame it was made, and only then evolves.

Código 32: src/gui/GolGlWidget.cpp — present the current board, then advance.

```
// Move the CURRENT board into the texture (interop or host-upload).
glBindTexture(GL_TEXTURE_2D, tex_);
if (present_ == Present::Interop && cudaEngine_) {
  interop_.copyFromDevice(cudaEngine_->currentDeviceBuffer());   // D2D into the PBO
  glBindBuffer(GL_PIXEL_UNPACK_BUFFER, pbo_);
  glTexSubImage2D(GL_TEXTURE_2D, 0, 0,0, cols, rows, GL_RED_INTEGER,
    ↪ GL_UNSIGNED_BYTE, nullptr);
} else {
  engine_->download(grid_);                                  // D2H (or host copy)
  glBindBuffer(GL_PIXEL_UNPACK_BUFFER, 0);
  glTexSubImage2D(GL_TEXTURE_2D, 0, 0,0, cols, rows, GL_RED_INTEGER,
    ↪ GL_UNSIGNED_BYTE, grid_.data());
}
// ... draw the fullscreen triangle, emit HUD stats ...

// Advance for the NEXT frame; --gens N auto-pauses on reaching the cap.
if (playing_ && engine_)
  for (int i = 0; i < speed_; ++i) {
    engine_->step(); ++generation_;
    if (cfg_.generations > 0 && generation_ >= cfg_.generations) { playing_ = false; break;
    ↪ }
  }
```

12.7. The shaders

The board is drawn as a textured **fullscreen triangle**: a single oversized triangle covers the screen with one draw call and *no vertex buffer* — the three positions come from `gl_VertexID`. This is the standard way to run a fragment shader over every pixel.

Código 33: src/gui/shaders/display.vert — fullscreen triangle, no VBO.

```
void main() {
    vec2 v[3] = vec2[3](vec2(-1.0,-1.0), vec2(3.0,-1.0), vec2(-1.0,3.0));
    gl_Position = vec4(v[gl_VertexID], 0.0, 1.0);
}
```

The fragment shader maps each screen pixel to a board cell (inverting the same zoom/pan transform the CPU uses for mouse picking), fetches the cell, and colours it. Zoom is pixels-per-cell (`uScale`); pan is the board cell shown at the screen centre (`uCenter`); a colour-mode uniform selects a flat alive/dead palette or a live-neighbour-count ramp (the latter samples the eight neighbours straight from the texture — free, no extra buffer).

Código 34: src/gui/shaders/display.frag — screen pixel to cell, then colour.

```
uniform usampler2D uBoard;     // GL_R8UI: one texel per cell (0/1)
uniform vec2  uViewSize;       // framebuffer size in px
```

```
uniform float uScale;         // pixels per cell (zoom)
uniform vec2  uCenter;        // board cell at the screen centre (pan)

void main() {
    vec2 screen = gl_FragCoord.xy;
    screen.y = uViewSize.y - screen.y;                    // flip: row 0 at top
    ivec2 cell = ivec2(floor(uCenter + (screen - 0.5 * uViewSize) / uScale));
    // ... out-of-board -> background colour ...
    uint alive = texelFetch(uBoard, cell, 0).r;
    fragColor = (alive == 1u) ? vec4(0.85, 0.95, 0.85, 1.0)
                              : vec4(0.10, 0.10, 0.12, 1.0);
}
```

The shaders are GLSL `#version 330` core — the viewer uses nothing past GL 3.3 (integer textures, `texelFetch`, VAOs), so 3.3 maximises the range of drivers it runs on. They are **read at runtime** from `src/gui/shaders/` (the path baked in at configure time), mirroring how the OpenCL backend loads `kernel.cl` — editable without recompiling.

12.8. Interaction — picking, painting, panning, zoom, reset

Mouse picking is the inverse of the shader's pixel → cell map, so what the cursor points at is exactly what the shader drew there. Left-drag paints (Shift erases) via `pokeCell`; middle/right-drag pans; the wheel zooms about the cursor.

Código 35: src/gui/GolGlWidget.cpp — widget pixel to cell, and painting.

```
QPointF GolGlWidget::screenToBoard(QPointF pos) const {      // inverse of the shader
    ↪ map
  const double dpr = devicePixelRatioF();
  return QPointF(center_.x() + (pos.x()*dpr - 0.5*width()*dpr)  / scale_,
                 center_.y() + (pos.y()*dpr - 0.5*height()*dpr) / scale_);
}
void GolGlWidget::paintAt(QPointF pos, unsigned char value) {
  const QPointF b = screenToBoard(pos);
  engine_->pokeCell((size_t)std::floor(b.x()), (size_t)std::floor(b.y()), value);
  update();
}
```

Reset restores the board to generation 0 of the current pattern. It needs its own snapshot because `grid_` is reused as the host-upload scratch buffer and is overwritten every frame; so a separate `initialGrid_` is captured at each generation-0 event (initial seed, `reseed`, `clear`, and `loadRle`) and restored on demand — so re-watching the same RLE costs a keystroke, not a re-open.

Código 36: src/gui/GolGlWidget.cpp — snapshot-based reset.

```
// initialGrid_ is set to grid_ at every generation-0 event.
void GolGlWidget::resetToInitial() {
  grid_ = initialGrid_;      // restore the snapshot
  engine_->upload(grid_);    // re-seed the engine
  generation_ = 0;
}
```

Patterns load through the existing `RleLoader` / `Pattern` pipeline (Section 10): `–rle PATH` seeds at startup, and an **Open RLE...** button opens a `QFileDialog` that stamps a pattern centred at runtime. The control panel's buttons and sliders are plain Qt widgets connected to the widget's public slots with `connect(...)`; a per-frame `statsChanged` signal updates the generation/FPS/zoom labels, and a one-shot `backendInfo` signal puts the active engine and present mode in the window title.

12.9. Portability, the Optimus caveat, and the launch wrapper

Because the CPU engine uses the host-upload path, `gol_gui` builds and runs **with no CUDA toolkit at all**; CUDA only adds the zero-copy path. This is the main portability win, and it is reflected in the build wiring (Section 13, Figure 5).

One environment caveat is worth recording because it is not obvious. The development machine is an **Optimus** laptop (NVIDIA dGPU + Intel iGPU), and on it OpenGL defaults to the *Intel* iGPU. Interop requires the GL context and CUDA to be on the *same* GPU, so on a default context `cudaGraphicsGLRegisterBuffer` fails — which is exactly the fallback-to-host-upload case above (the viewer still runs, just with the PCIe round trip). To get the zero-copy path, the GL context must be placed on the NVIDIA GPU with PRIME render offload; `scripts/run_gui.sh` does this:

Código 37: scripts/run_gui.sh — put the GL context on the NVIDIA GPU (Optimus).

```
exec env __NV_PRIME_RENDER_OFFLOAD=1 __GLX_VENDOR_LIBRARY_NAME
    ↪ =nvidia \
      QT_QPA_PLATFORM=xcb "$bin" "$@"
```

`QT_QPA_PLATFORM=xcb` routes through Xwayland so the GLX offload variables apply on a Wayland session. On a single-GPU desktop none of this is needed — a bare `./gol_gui` already gets an NVIDIA (or AMD/Intel) context and the right path is chosen automatically.

12.10. Build wiring and the command line

`gol_gui` is gated on `BUILD_GUI` and is independent of `BUILD_CUDA`: it always links the core and the CPU engine, and only when the CUDA engine is configured does it also build the

gol_cudagl interop bridge (nvcc), link the CUDA engine, and define GOL_HAVE_CUDA (which guards every CUDA reference in the widget).

Código 38: CMakeLists.txt — the viewer target; CUDA is optional.

```
option(BUILD_GUI "Build the Qt OpenGL viewer" OFF)
if(BUILD_GUI)
  find_package(Qt6 REQUIRED COMPONENTS Core Gui Widgets OpenGL
    ↪ OpenGLWidgets)
  set(_gui_libs gol_core gol_cpu
              Qt6::Core Qt6::Gui Qt6::Widgets Qt6::OpenGL Qt6::OpenGLWidgets)
  if(GOL_CUDA_ENABLED)                              # zero-copy path, optional
    find_package(CUDAToolkit REQUIRED)
    add_library(gol_cudagl src/render/CudaGlInterop.cu)  # compiled by nvcc
    list(APPEND _gui_libs gol_cuda gol_cudagl CUDA::cudart)
    list(APPEND _gui_defs GOL_HAVE_CUDA)
  endif()
  add_executable(gol_gui src/gui/main_gui.cpp src/gui/GolGlWidget.cpp
              src/gui/MainWindow.cpp <headers for AUTOMOC>)
  set_target_properties(gol_gui PROPERTIES AUTOMOC ON)
  target_link_libraries(gol_gui PRIVATE ${_gui_libs})
endif()
```

The Q_OBJECT headers live apart from their .cpp (in include/gol/gui/), so they are listed explicitly in the target for AUTOMOC to scan — it only auto-pairs same-directory basenames. The CLI mirrors the headless gol flags that define *what* to simulate (–rows/–cols/–gens/–threads/–wrap/–seed/–rle/–engine cpu|cuda/–block, plus a COLSxROWS shorthand); the headless-only flags (–renderer, –csv, –verify) do not apply, and –gens becomes an interactive auto-pause rather than an exit condition.

13. Build system — CMakeLists.txt

The build must produce a working CPU + text binary on a machine with no CUDA toolkit and no OpenCL. That requirement shapes the GPU targets.

Código 39: CMakeLists.txt — the core/CPU targets and the GPU double-gate.

```
add_library(gol_core STATIC
   src/core/Config.cpp src/patterns/Pattern.cpp src/patterns/RleLoader.cpp)
target_include_directories(gol_core PUBLIC ${CMAKE_CURRENT_SOURCE_DIR}/
    ↪ include)

find_package(Threads REQUIRED)                  # std::thread / std::barrier
add_library(gol_cpu src/engines/cpu/CpuEngine.cpp)
target_link_libraries(gol_cpu PUBLIC gol_core Threads::Threads)

```

```
add_executable(gol src/core/main.cpp)
target_link_libraries(gol PRIVATE gol_core gol_cpu gol_render)

option(BUILD_CUDA "Build the CUDA engine" OFF)
if(BUILD_CUDA)
   set(_cuda_src ${CMAKE_CURRENT_SOURCE_DIR}/src/engines/cuda/CudaEngine.
    ↪ cu)
   if(EXISTS ${_cuda_src})                    # second gate: source must exist
      enable_language(CUDA)
      add_library(gol_cuda ${_cuda_src} .../kernel.cu)
      target_link_libraries(gol_cuda PUBLIC gol_core)
   else()
      message(WARNING "BUILD_CUDA=ON but source does not exist yet; skipping.")
   endif()
endif()
```

Each GPU backend is **double-gated**: an `option(... OFF)` *and* an `if(EXISTS <source>)` check. Even passing `-DBUILD_CUDA=ON` today emits a warning and skips the target because the `.cu` source does not exist yet — it does not fail configuration. The OpenCL target follows the identical pattern. `find_package(Threads REQUIRED)` is what makes `std::thread`/`std::barrier` link on the CPU engine.

Código 40: Building and testing.

```
cmake -S . -B build              # core + CPU + tests (GPU OFF by default)
cmake --build build
ctest --test-dir build --output-on-failure
# GPU opt-in (once the sources exist):
cmake -S . -B build -DBUILD_CUDA=ON -DBUILD_OPENCL=ON
```

The test `CMakeLists` prefers a system GoogleTest via `find_package(GTest CONFIG)` and otherwise `FetchContents` v1.17.0; it defines `PATTERNS_DIR` so tests find the RLE fixtures regardless of working directory, and registers each test with `gtest_discover_tests`.

14. Tests — the executable specification of the invariants

The tests are not an afterthought; they encode the invariants from Section 2 as runnable checks, in dependency order (`CpuEngine` is the oracle, so it is verified first).

- **`compile_smoke_test.cpp`** includes *every* interface header (so the scaffolding stays build-clean and mutually consistent) but deliberately does not call the declared-but-undefined symbols, so it links without those translation units. Its `VariantRule` case pins the rule truth table directly.

- **rules_test.cpp** checks CpuEngine on known patterns: a block is a still-life, a blinker oscillates with period 2, and — the defining case — a dead cell with exactly six live neighbours is born.
- **cpu_parallel_test.cpp** proves the central design claim. On a deliberately non-square, non-thread-divisible 128×130 grid it asserts thread counts $\{2, 3, 4, 8, 0\}$ all match the single-thread reference *bit-for-bit*, in both edge modes; a second case targets rangeFor's remainder logic with 17×9 and counts $4, 5$.
- **rle_loader_test.cpp** covers parsing, dimensions, specific coordinates (including that birth_on_six's centre is dead), the missing-file throw, and stamping with clipping.

Código 41: tests/cpu_parallel_test.cpp — the equivalence assertion.

```
const Grid reference = runN(start, 1, wrap, 25);        // sequential oracle
for (unsigned threads : {2u, 3u, 4u, 8u, 0u /* all hw cores */}) {
  Grid parallel = runN(start, threads, wrap, 25);
  EXPECT_TRUE(parallel == reference)
     << "threads=" << threads << " wrap=" << wrap << " diverged";
}
```

The fourth test: cross-backend equivalence.

Done for both GPU backends. cuda_equivalence_test.cpp and opencl_equivalence_test.cpp seed their engine and the CpuEngine oracle identically and assert bit-for-bit equality across block sizes $\{32, 64, 128, 256\}$, both kernels (global/shared) and both edge modes; each also checks the host $\leftrightarrow$ device round-trip (upload then download, no step) and buffer reuse across grid sizes on one engine instance. An external oracle is also available: the headless bgolly runner simulates B36/S23 on an infinite grid, and the highlife_replicator.rle / highlife_spaceship.rle fixtures already reproduce it bit-for-bit, so they make good large-grid cross-checks for the GPU backends.

15. Timing methodology and the expected GPU story

15.1. The metric

The headline number is fixed across all three backends:

$$\text{Mcells/s} = \frac{\text{rows} \times \text{cols} \times \text{generations}}{t_{\text{elapsed}} \times 10^6} \tag{1}$$

measured against NullRenderer so output and transfer cost never enter t. Timing is uniform and in-program (not from a profiler) so the three backends are measured the same

way, exposed through `ISimEngine::lastKernelMillis()`: `std::chrono` on CPU, `cudaEvent_t` around the launch on CUDA, and command-queue events with `CL_QUEUE_PROFILING_ENABLE` on OpenCL. Kernel time is reported separately from host ↔ device transfers.

For the sweep data the binary has a CSV mode: `--csv` prints one data row (columns `backend, rows, cols, generations, threads, block, shared, wrap, kernel_ms, wall_ms, mcells_kernel, mcells_wall`) and `--csv-header` prints just the header and exits, so a script writes the header once and appends a row per configuration straight into Pandas. In practice that script is `scripts/sweep.sh` (the CPU-threads and CUDA/OpenCL block × shared/local sweeps, with an adaptive-repeats runtime guard), and `analysis/results.ipynb` turns the resulting `results/linux/*.csv` into the report's figures. The speed-up is

$$S(N) = \frac{T_{\mathrm{CPU}}(N)}{T_{\mathrm{GPU}}(N)}. \tag{2}$$

15.2. What the numbers should show, and why

This is a memory-bound, arithmetically trivial nine-point stencil over one-byte cells. That single fact predicts the shape of every curve the report measures (and the profiling then refines):

Block size gives a concave curve peaking at 64–128.

Below that, blocks are too small to hide memory latency (too few warps in flight to cover the load-use stalls); past it, more threads per block do not buy more occupancy, so the curve flattens and then falls (512). The mechanism is occupancy and latency-hiding, not arithmetic — the profiling confirms the kernel is latency-bound on cache-served loads, not bandwidth-bound on DRAM.

Shared/local memory loses here.

The classic stencil optimisation stages a tile plus a halo into shared memory to reuse neighbour reads. Here each cell is one byte and the L2 already serves the reuse, so the staging saves no DRAM traffic (≈ 2 B/cell either way) while adding a barrier and dropping L1 reuse — it runs ≈ 0.5× the global kernel (a ≈ 1.8–2.1× penalty), on both CUDA and OpenCL. Explaining *why* is one of the more interesting parts of the report.

Launch overhead dominates at small grids.

With few cells there are too few blocks to fill the SMs, so per-step launch cost dominates: the GPU's `wall/kernel` efficiency falls (to ≈ 26–40% at $N = 64$, lower for OpenCL whose faster kernel makes the fixed launch cost weigh more), recovering by $N \approx 1024$–2048. Because the metric times the kernel/loop with the `NullRenderer` (transfers excluded), the GPU still outruns the compute-bound CPU at every measured N; a classic $S(N) < 1$ regime appears only once the PCIe transfer cost is counted. The sweeps therefore run from small to large $N \times M$ to expose both the efficiency knee and the asymptote.

The deep-dive profiling targets the *best* parallel solution only: `ncu` (Nsight Compute) for CUDA kernel-internal metrics — occupancy, stalls, memory throughput (it will not profile OpenCL) — and `nsys` (Nsight Systems) for the system timeline, which can also trace OpenCL

on NVIDIA hardware.

16. Status, roadmap, and how to extend

Done.

The backend-agnostic core, the CPU engine (sequential and parallel in one path), both renderers, the RLE pipeline, the CLI, the `CudaEngine` (global + shared kernels, `−block`/`−shared`, cudaEvent timing) and the `OpenCLEngine` (its `life_global`/`life_local` twin, runtime-loaded `kernel.cl`, queue-event timing; Section 9), and the CSV benchmark-output mode (`−csv`/`−csv-header`). Test suites: CPU rules, sequential-vs-parallel, RLE loader, and the cross-backend equivalence of *both* GPU engines against the CPU oracle (Sections 8, 9). The CPU engine is the reference oracle. The benchmark sweeps (`scripts/sweep.sh`, now CPU + CUDA + OpenCL), the results CSVs, the analysis notebook (`analysis/results.ipynb`), and the Nsight (`ncu`/`nsys`) profiling of the best CUDA kernel are also done and populate `main.tex`. The interactive **Qt + OpenGL viewer** (`gol_gui`) is also done (Section 12): CUDA ↔ OpenGL zero-copy interop with a host-upload fallback, the CPU engine supported with no CUDA toolkit, live paint/zoom/pan, RLE loading, and reset.

Pending / future work.

All three backends are implemented, verified against the CPU oracle (including `gol_opencl_tests`) and benchmarked — the OpenCL results are folded into the report alongside CUDA. The interactive viewer realizes what used to be the "planned `GuiRenderer`", though as a separate front-end rather than an `IRenderer` (Section 12 explains why). What is left is optional: shader-side visualisation extensions (age/heat colouring, a population plot, a 3-D height field via a geometry shader), and the performance ideas the report raises (a vectorised AVX2 CPU path; more instruction-level parallelism per GPU thread to hide load latency). Nsight Compute profiles only CUDA, so OpenCL relies on its in-program queue-event timing.

How a GPU backend slots in.

Implement `ISimEngine`: own two device buffers, `upload` once, ping-pong on the device inside `step`, `download` only when rendering. Reuse `nextState` from `LifeRules.hpp` (CUDA includes it directly; OpenCL mirrors it in `kernel.cl`). Match the edge handling of `countNeighbors` exactly, in both bounded and toroidal modes. Wire it into `main.cpp`'s engine selection and add the cross-backend test against the CPU oracle. Because the `Grid` layout and the metric are already fixed, a correct new backend needs no changes to the core, the harness, or the report's measurement code.

17. Keeping this manual in sync

This manual, the graded report (`main.tex`), and `CLAUDE.md` are treated as living documents. The rule, recorded in `CLAUDE.md` under "Documentation maintenance", is: whenever a source file is added, removed, or changed in a way that alters behaviour or structure,

update all three in the same change — this manual's relevant section and file map (Table 1), the report if results or methodology move, and `CLAUDE.md`'s "Current state" and layout. The most common trigger will be landing the CUDA or OpenCL backend: that touches Sections 4.2, 7 (as the oracle reference), 14, 15, and 16 here, plus the implementation and results sections of the report.